

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA0302	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	T. Bhanu Pavan Varma	Reg. No.:	192425380
Section / Batch:	B. Tech AI&ML 2024-2028	Date:	13/07/2026

Code of Conduct

I T. Bhanu Pavan Varma(192425380) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.

Identify email addresses.

Extract mobile numbers.

Detect technical skills (Python, Java, SQL, Machine Learning, NLP).

Extract years of experience.

Generate a structured summary of the candidate's profile.

Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Aim

To design and implement a Python-based Resume Information Extraction System using **Regular Expressions (Regex)** that extracts candidate details such as **name, email address, mobile number, technical skills, and years of experience**, generates a structured candidate profile summary, and shortlists candidates who satisfy the minimum eligibility criteria (at least **2 years of experience** and **Python** skill).

Procedure

1. Import the **re (Regular Expressions)** module in Python.
2. Create sample resume data in text format.
3. Define the list of technical skills to search for:
 - Python
 - Java
 - SQL
 - Machine Learning
 - NLP
4. Use Regular Expressions to extract:
 - Candidate Name
 - Email Address
 - Mobile Number
 - Years of Experience
5. Search the resume text for predefined technical skills.
6. Generate a structured summary containing all extracted information.
7. Check the eligibility criteria:
 - Experience should be **2 years or more**.
 - Candidate should possess **Python** skill.
8. Display the list of eligible candidates.

Code

```
import re

# -----
# Sample Resume Data
# -----

resumes = [
    """
    Name: John Smith
    Email: johnsmith@gmail.com
    Phone: +91-9876543210

    Skills:
    Python, SQL, Machine Learning, NLP

    Experience:
    3 years of experience in Data Science.
    """,

    """
    Name: Alice Johnson
    Email: alice.johnson@yahoo.com
    Phone: 9123456789

    Skills:
    Java, SQL

    Experience:
    1 year of experience in Software Development.
    """,

    """
    Name: Robert Brown
    Email: robert.brown@outlook.com
    Phone: 9876512345

    Skills:
    Python, Java, SQL

    Experience:
    5 years experience in Full Stack Development.
    """
]
```

```
technical_skills = [  
    "Python",  
    "Java",  
    "SQL",  
    "Machine Learning",  
    "NLP"  
]
```

```
eligible_candidates = []
```

```
for resume in resumes:
```

```
    print("=" * 60)
```

```
    # Extract Name
```

```
    name = re.search(r"Name:\s*(.*)", resume)
```

```
    # Extract Email
```

```
    email = re.search(  
        r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}",  
        resume  
    )
```

```
    # Extract Mobile Number
```

```
    phone = re.search(  
        r"(\+91[- ])?[6-9]\d{9}",  
        resume  
    )
```

```
    # Extract Skills
```

```
    skills_found = []
```

```
    for skill in technical_skills:
```

```
        if re.search(skill, resume, re.IGNORECASE):  
            skills_found.append(skill)
```

```
    # Extract Experience
```

```
    experience = re.search(  
        r"(\d+)\s+year",  
        resume,  
        re.IGNORECASE  
    )
```

```
    years = int(experience.group(1)) if experience else 0
```

```

# Display Candidate Summary
print("Candidate Summary")
print("-----")
print("Name      :", name.group(1) if name else "Not Found")
print("Email      :", email.group() if email else "Not Found")
print("Phone      :", phone.group() if phone else "Not Found")
print("Skills     :", ", ".join(skills_found))
print("Experience :", years, "Years")

# Eligibility Check
if years >= 2 and "Python" in skills_found:
    eligible_candidates.append(name.group(1))

print("\n")
print("=" * 60)
print("Eligible Candidates")
print("=" * 60)

for candidate in eligible_candidates:
    print(candidate)

```

Output

Candidate Summary

```

-----
Name      : John Smith
Email      : johnsmith@gmail.com
Phone      : +91-9876543210
Skills     : Python, SQL, Machine Learning, NLP
Experience : 3 Years

```

Candidate Summary

```

-----
Name      : Alice Johnson
Email      : alice.johnson@yahoo.com
Phone      : 9123456789
Skills     : Java, SQL
Experience : 1 Years

```

Candidate Summary

Name : Robert Brown
Email : robert.brown@outlook.com
Phone : 9876512345
Skills : Python, Java, SQL
Experience : 5 Years

Eligible Candidates

John Smith
Robert Brown

Result

The Python program was successfully implemented using **Regular Expressions (Regex)** to extract candidate information from resumes. The system accurately identified the **candidate's name, email address, mobile number, technical skills, and years of experience**, generated a structured summary for each resume, and shortlisted candidates based on the specified eligibility criteria (**minimum 2 years of experience and Python skill**). The eligible candidates identified were **John Smith** and **Robert Brown**.

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.
Perform prefix-based searches.
Perform suffix-based searches.
Search using partial keywords.
Perform case-insensitive searches.
Display all matching product names.
Generate a report showing the total number of matching products for each search.

Aim

To design and implement a **Python-based Product Search System using Regular Expressions (Regex)** that enables **exact keyword search, prefix search, suffix search, partial keyword search, and case-insensitive search**, displays all matching product names, and generates a report showing the total number of matching products for each search.

Procedure

1. Import the **re (Regular Expressions)** module.
2. Create a list of sample product names.
3. Accept search keywords for different search operations.
4. Use Regular Expressions to perform:
 - Exact keyword search.
 - Prefix-based search.
 - Suffix-based search.
 - Partial keyword search.
 - Case-insensitive search.
5. Store all matching product names.
6. Display the matching products for each search.
7. Count the number of matched products.
8. Generate a report showing the total matches for each search type.

Code

```
import re
```

```
# -----
```

```
# Product List
```

```
# -----
```

```
products = [  
    "Apple iPhone 15",  
    "Apple Watch",  
    "Samsung Galaxy S24",  
    "Samsung TV",  
    "Dell Laptop",  
    "HP Laptop",  
    "Lenovo Laptop",  
    "Apple AirPods",  
    "Boat Headphones",  
    "Sony Headphones",  
    "Noise Smart Watch",  
    "Canon Camera",  
    "Nikon Camera"  
]
```

```
# -----
```

```
# Search Functions
```

```
# -----
```

```
def search_products(pattern, flags=0):  
    matches = []  
    for product in products:  
        if re.search(pattern, product, flags):  
            matches.append(product)  
    return matches
```

```
# -----
```

```
# Exact Keyword Search
```

```
# -----
```

```
exact = search_products(r"\bLaptop\b")
```

```
# -----
```

```
# Prefix Search
```

```
# -----
```

```
prefix = search_products(r"^Apple")
```

```
# -----
```

```
# Suffix Search
```

```
# -----
```

```
suffix = search_products(r"Camera$")
```



```

# -----
# Partial Keyword Search
# -----

partial = search_products(r"Watch")

# -----
# Case-Insensitive Search
# -----

case_insensitive = search_products(r"headphones", re.IGNORECASE)

# -----
# Display Results
# -----

print("="*60)
print("Exact Keyword Search (Laptop)")
print(exact)

print("\n"+"="*60)
print("Prefix Search (Apple)")
print(prefix)

print("\n"+"="*60)
print("Suffix Search (Camera)")
print(suffix)

print("\n"+"="*60)
print("Partial Keyword Search (Watch)")
print(partial)

print("\n"+"="*60)
print("Case-Insensitive Search (headphones)")
print(case_insensitive)

print("\n")
print("="*60)
print("Search Report")
print("="*60)

print("Exact Search Matches      :", len(exact))
print("Prefix Search Matches      :", len(prefix))
print("Suffix Search Matches       :", len(suffix))
print("Partial Search Matches      :", len(partial))

```

```
print("Case-Insensitive Matches : ", len(case_insensitive))
```

Output

Exact Keyword Search (Laptop)

['Dell Laptop', 'HP Laptop', 'Lenovo Laptop']

Prefix Search (Apple)

['Apple iPhone 15', 'Apple Watch', 'Apple AirPods']

Suffix Search (Camera)

['Canon Camera', 'Nikon Camera']

Partial Keyword Search (Watch)

['Apple Watch', 'Noise Smart Watch']

Case-Insensitive Search (headphones)

['Boat Headphones', 'Sony Headphones']

Search Report

Exact Search Matches	: 3
Prefix Search Matches	: 3
Suffix Search Matches	: 2
Partial Search Matches	: 2
Case-Insensitive Matches	: 2

Result

The **Python-based Product Search System** was successfully implemented using **Regular Expressions (Regex)**. The program efficiently performed **exact keyword, prefix, suffix, partial keyword, and case-insensitive searches**, displayed all matching product names, and generated a report showing the total number of matching products for each search type. The system demonstrates how Regex can be effectively used to build flexible and efficient product search functionality in e-commerce applications.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.
Validate the institutional email address.
Validate the course code.
Validate the semester information.
Validate the student's mobile number.
Display appropriate validation messages for each field.
Generate a final registration status report indicating whether the registration is successful.

Aim

To design and implement a **Python-based Student Registration Validation System using Regular Expressions (Regex)** that validates the **student register number, institutional email address, course code, semester information, and mobile number**, displays appropriate validation messages for each field, and generates a final registration status report indicating whether the registration is successful.

Procedure

1. Import the **re (Regular Expressions)** module.
2. Accept or define sample student registration details.
3. Use Regular Expressions to validate:
 - Student Register Number
 - Institutional Email Address
 - Course Code
 - Semester
 - Mobile Number
4. Display a validation message for each field.
5. Check whether all fields are valid.
6. If all validations are successful, display **Registration Successful**; otherwise, display **Registration Failed**.
7. Generate the final registration status report.

Code

```
import re
```

```
# -----  
# Student Details  
# -----
```

```
register_number = "23CSE1015"
email = "student@university.edu"
course_code = "CS301"
semester = "Semester 5"
mobile = "9876543210"
```

```
registration_success = True
```

```
if re.fullmatch(r"\d{2}[A-Z]{3}\d{4}", register_number):
    print("Register Number : Valid")
else:
    print("Register Number : Invalid")
    registration_success = False
```

```
# -----
# Institutional Email Validation
# Example: student@university.edu
# -----
```

```
if re.fullmatch(r"[A-Za-z0-9._%+-]+@university\.edu", email):
    print("Email      : Valid")
else:
    print("Email      : Invalid")
    registration_success = False
```

```
# -----
# Course Code Validation
# Format: 2 letters + 3 digits
# Example: CS301
# -----
```

```
if re.fullmatch(r"[A-Z]{2}\d{3}", course_code):
    print("Course Code  : Valid")
else:
    print("Course Code  : Invalid")
    registration_success = False
```

```
# -----
# Semester Validation
# Semester 1 to Semester 8
# -----
```

```
if re.fullmatch(r"Semester [1-8]", semester):
    print("Semester    : Valid")
else:
```

```

print("Semester      : Invalid")
registration_success = False

if re.fullmatch(r"[6-9]\d{9}", mobile):
    print("Mobile Number  : Valid")
else:
    print("Mobile Number  : Invalid")
    registration_success = False

print("\n" + "=" * 50)
print("Registration Status Report")
print("=" * 50)

if registration_success:
    print("Registration Status : SUCCESSFUL")
else:
    print("Registration Status : FAILED")

```

Output

```

Register Number : Valid
Email           : Valid
Course Code     : Valid
Semester        : Valid
Mobile Number   : Valid

```

```

=====
Registration Status Report
=====

```

```

Registration Status : SUCCESSFUL

```

Result

The **Python-based Student Registration Validation System** was successfully implemented using **Regular Expressions (Regex)**. The program correctly validated the **student register number, institutional email address, course code, semester information, and mobile number**, displayed appropriate validation messages for each field, and generated a final registration status report. Since all input fields were valid, the registration was completed successfully.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____